

# Reshaping: `pivot_longer` · `pivot_wider`

Thursday, June 4, 2026

 Today: Thursday June 4

## Before class

- Perusall Assignment: [DC §12.1](#)

## Plan for today

- What makes data **tidy**, and why the tidyverse wants it that way
- **Wide** vs. **narrow (long)** layouts of the *same* data
- `pivot_longer()` and `pivot_wider()`
- Choosing a shape to fit the task

## Helpful links

- [Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 5](#) · [DC Ch. 12](#) · [tidyr reference](#)

## Recap, and what's new

All week you've manipulated data **within** its existing shape, choosing rows, columns, and groups.

### Tip

`pivot_longer()` and `pivot_wider()` change the **shape** of the table itself, without changing any of the values.

These come from **tidyr**, dplyr's sibling package (both load with `library(tidyverse)`).

## Tidy data

### What “tidy” means

A dataset is **tidy** when:

1. Each **variable** is a column.
2. Each **observation** is a row.

The same numbers can be stored many different ways. Only one of the following is tidy:

```
table1 # tidy: country, year, cases, population each a column
```

```
# A tibble: 6 x 4
  country    year cases population
  <chr>    <dbl> <dbl>    <dbl>
1 Afghanistan 1999    745  19987071
2 Afghanistan 2000   2666  20595360
3 Brazil      1999  37737  172006362
4 Brazil      2000  80488  174504898
5 China       1999 212258 1272915272
6 China       2000 213766 1280428583
```

### The same data, untidy

```
table2 # cases and population are stacked into type/count
```

```
# A tibble: 12 x 4
  country    year type      count
  <chr>    <dbl> <chr>    <dbl>
1 Afghanistan 1999 cases      745
2 Afghanistan 1999 population 19987071
3 Afghanistan 2000 cases      2666
4 Afghanistan 2000 population 20595360
5 Brazil      1999 cases      37737
6 Brazil      1999 population 172006362
7 Brazil      2000 cases      80488
8 Brazil      2000 population 174504898
9 China       1999 cases     212258
10 China       1999 population 1272915272
```

|          |                 |            |
|----------|-----------------|------------|
| 11 China | 2000 cases      | 213766     |
| 12 China | 2000 population | 1280428583 |

`table2` holds identical information, but `type/count` jams two variables into a name/value pair. Computing a rate is awkward here and natural in `table1`.

The dead giveaway that the data is **not** tidy here is that there are multiple rows corresponding to the same country / year observation.

#### **i** Note

Why tidy? `dplyr`, `ggplot2`, and friends are built for tidy data. Tidy data is the shape where `mutate()`, `group_by()`, and `aes()` work without needing to change the shape of the data first.

## Wide vs. narrow

### Two layouts of one dataset

Reshaping refers to moving back and forth between **wide** and **narrow** data formats. For example, data on blood pressure (BP) in wide form:

```
bp_wide <- tribble(
  ~subject, ~before, ~after,
  "BHO",    160,    160,
  "GWB",    115,    135,
  "WJC",    105,    145
)
bp_wide
```

```
# A tibble: 3 x 3
  subject before after
  <chr>    <dbl> <dbl>
1 BHO      160   160
2 GWB      115   135
3 WJC      105   145
```

Here, we have row per **subject**, and **before/after** are separate columns.

## The same BP data, narrow

```
bp_narrow <- bp_wide |>
  pivot_longer(
    cols      = c(before, after),
    names_to  = "when",
    values_to = "sbp"
  )
bp_narrow
```

```
# A tibble: 6 x 3
  subject when      sbp
  <chr>   <chr>  <dbl>
1 BHO    before   160
2 BHO    after    160
3 GWB    before   115
4 GWB    after    135
5 WJC    before   105
6 WJC    after    145
```

Now one row per **measurement occasion** (before/after): the case changed meaning, the numbers didn't.

### Tip

Each shape has strengths. *Wide* makes a before/after change a one-line `mutate(change = after - before)`. *Narrow* makes it easy to add more measurement occasions or more variables, and it's what `ggplot2` usually expects.

## pivot\_longer()

### The idea

`pivot_longer()` takes several columns and stacks them into **two** new columns: one holding the old **names**, one holding the old **values**.

There are three arguments after the data:

- `cols`: which columns to gather (compatible with `select()` syntax: `starts_with()`, ranges with `:`, `!`)

- `names_to`: name for the new column that will hold the **old column names**
- `values_to`: name for the new column that will hold the **cell values**

```
bp_wide |>
  pivot_longer(cols = c(before, after), names_to = "when", values_to = "sbp")
```

#### Warning

Note that `names_to` and `values_to` are **quoted**. They're names you're inventing, so they don't exist in the data yet.

### A real one: when names ARE data

The `billboard` data (in `tidyr`) has one column **per week** (`wk1`, `wk2`, ...). The week is a variable trapped in the column names:

```
billboard |>
  pivot_longer(
    cols      = starts_with("wk"),
    names_to  = "week",
    values_to = "rank",
    values_drop_na = TRUE
  )
```

# A tibble: 5,307 x 5

|    | artist  | track                   | date.entered | week  | rank  |
|----|---------|-------------------------|--------------|-------|-------|
|    | <chr>   | <chr>                   | <date>       | <chr> | <dbl> |
| 1  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk1   | 87    |
| 2  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk2   | 82    |
| 3  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk3   | 72    |
| 4  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk4   | 77    |
| 5  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk5   | 87    |
| 6  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk6   | 94    |
| 7  | 2 Pac   | Baby Don't Cry (Keep... | 2000-02-26   | wk7   | 99    |
| 8  | 2Ge+her | The Hardest Part Of ... | 2000-09-02   | wk1   | 91    |
| 9  | 2Ge+her | The Hardest Part Of ... | 2000-09-02   | wk2   | 87    |
| 10 | 2Ge+her | The Hardest Part Of ... | 2000-09-02   | wk3   | 92    |

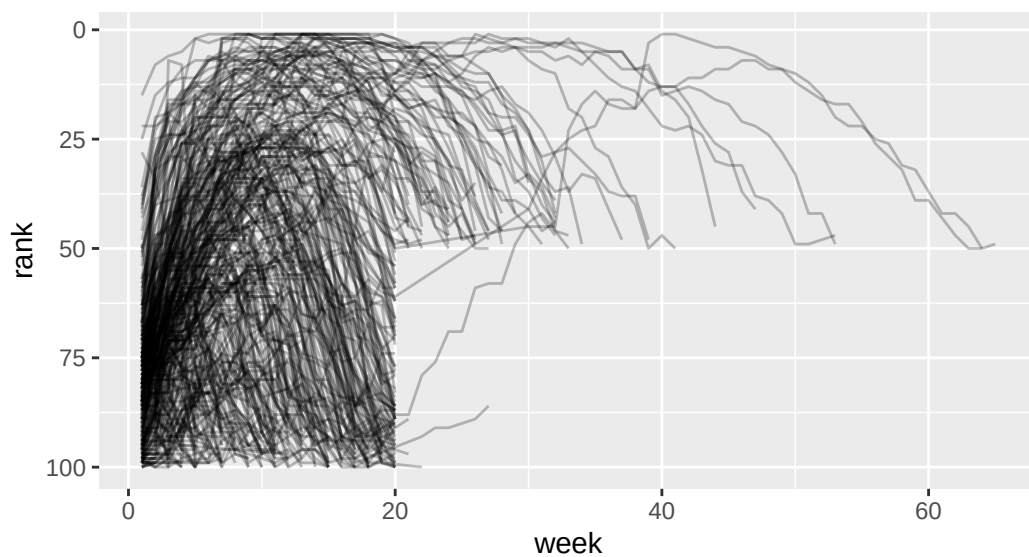
# i 5,297 more rows

`values_drop_na = TRUE` drops the empty weeks (songs that had already fallen off the chart).

## Now it's plottable

In wide form you *couldn't* put “week” on an axis, it wasn't a variable. After pivoting, it is:

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week", values_to = "rank",
    values_drop_na = TRUE
  ) |>
  mutate(week = parse_number(week)) |> # "wk3" -> 3
  ggplot(aes(x = week, y = rank, group = track)) +
  geom_line(alpha = 0.25) +
  scale_y_reverse() # rank 1 on top
```



### **i** Note

This is *why* we pivot: getting a variable out of the column names and into a real column is often the step that makes a plot possible. (`parse_number()` pulls the number out of "wk3".)

## pivot\_wider()

### The opposite move

`pivot_wider()` spreads one column's values across **new columns**. Its two key arguments are the opposite of before:

- `names_from`: the column whose **values become new column names**
- `values_from`: the column whose values **fill the new cells**

```
bp_narrow |>
  pivot_wider(names_from = when, values_from = sbp)
```

```
# A tibble: 3 x 3
  subject before after
  <chr>      <dbl> <dbl>
1 BHO         160   160
2 GWB         115   135
3 WJC         105   145
```

We're back to `bp_wide`. `pivot_longer()` and `pivot_wider()` are inverses.

### When you might use it

Previously we learned `group_by()` |> `summarise()` produces a **long** summary. Widening it makes a readable comparison table.

```
penguins |>
  filter(!is.na(sex)) |>
  group_by(species, sex) |>
  summarise(avg_mass = mean(body_mass_g, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = sex, values_from = avg_mass)
```

```
# A tibble: 3 x 3
  species female male
  <fct>      <dbl> <dbl>
1 Adelie   3369. 4043.
2 Chinstrap 3527. 3939.
3 Gentoo   4680. 5485.
```

## Comparison becomes a mutate operation

With `female` and `male` now side by side in each row, the difference is one `mutate()`:

```
penguins |>
  filter(!is.na(sex)) |>
  group_by(species, sex) |>
  summarise(avg_mass = mean(body_mass_g, na.rm = TRUE)) |>
  pivot_wider(names_from = sex, values_from = avg_mass) |>
  mutate(diff = male - female)
```

```
# A tibble: 3 x 4
# Groups:   species [3]
  species female male diff
<fct>     <dbl> <dbl> <dbl>
1 Adelie   3369. 4043.  675.
2 Chinstrap 3527. 3939.  412.
3 Gentoo   4680. 5485.  805.
```



Tip

Long form is for **computing and plotting across** a category; wide form is for **reading and comparing within** a row.

## Choosing a shape

### Which way do I pivot?

Table 1: Match the shape to the next step.

| Your goal                                 | Shape you want | Verb to get there           |
|-------------------------------------------|----------------|-----------------------------|
| Put a “trapped” variable on a plot axis   | narrow / long  | <code>pivot_longer()</code> |
| Group or summarise over a category        | narrow / long  | <code>pivot_longer()</code> |
| Compare two values <b>per row</b> (a – b) | wide           | <code>pivot_wider()</code>  |
| A human-readable comparison table         | wide           | <code>pivot_wider()</code>  |



### Note

There isn't always one "tidy" shape, it depends what you're doing next. It's helpful to be comfortable with **untidying, transforming, and re-tidying** as the task changes.

## What we covered today

- **Tidy data:** variables in columns, observations in rows, values in cells
- **Wide vs. narrow:** the same values, different meaning of a "row"
- **pivot\_longer():** gather many columns into **names\_to + values\_to**; the fix when a variable is hiding in column names
- **pivot\_wider():** spread **names\_from + values\_from** into new columns; the way to a comparison table
- **They're inverses**, and the right shape is whichever makes your *next* step easiest

## Activity

### Activity: reshaping

Roughly 25 minutes. Individually or with a neighbor.

- [Activity Canvas Link](#)

**Format:** you'll diagnose whether tables are tidy/wide/narrow, pivot a tiny table by hand-prediction then check, pivot **billboard** longer and plot it, widen a **penguins** summary into a comparison table, confirm the two verbs undo each other, and debug a pivot that went "too narrow."

**To turn in:** save your `.qmd` and rendered HTML.

## Wrap-up & looking ahead

**Tomorrow (Friday): Quiz 3** in the first 20 minutes, then version control, Git and GitHub concepts and a demonstration, so your project team can collaborate on shared code.

### Upcoming Deadlines

- **Tonight, Thu 6/4, 11:59 p.m.:** [Project: Team formation](#)
- **Fri 6/5, before class:** [DC §9.1–9.2](#)
- **Fri 6/5, in class:** Quiz 3
- **Fri 6/5, 11:59 p.m.:** HW 3

## Reach out

Stuck? Office hours, or email me ([cgc5478@psu.edu](mailto:cgc5478@psu.edu)) or Xinyue ([xpw5228@psu.edu](mailto:xpw5228@psu.edu)).

## Reference

[Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 5](#) · [DC Ch. 12](#) · [tidyr reference](#)